

Integrating Reinforcement Learning into Pipe System Design

Ing. David Fait
Department of Machine Design
University of West Bohemia
Pilsen, Czech Republic
dfait@fst.zcu.cz, 0000-0003-3552-1343

Ing. Petra Hofrichterová
Department of Machine Design
University of West Bohemia
Pilsen, Czech Republic
hofrichp@fst.zcu.cz, 0000-0003-2389-1830

Ing. Václav Mašek
Department of Machine Design
University of West Bohemia
Pilsen, Czech Republic
vmasek@fst.zcu.cz, 0000-0002-0567-4835

Abstract—The purpose of this paper is to describe one step in the broader research aimed at the design of machines using modern methods of artificial intelligence. This article explores the integration of reinforcement learning into pipe system design. The goal is to partially substitute the work of human designers. While integrating reinforcement learning into pipe system design may help streamline the process, it is important to consider the limitations and potential biases that could arise from relying solely on artificial intelligence. Therefore, the program must be robust enough to handle a variety of scenarios that may occur during the design process. The first part introduces the topic and provides an overview of the current state-of-the-art techniques. Later, the basic functionality of the neural networks and, in particular, reinforcement learning algorithms is discussed. The paper then proceeds with the application of machine design and introduces the presented solution and its main parts. The paper then concludes with the advantages of this approach, a description of its functionality and its limits. The very last part presents the authors' future plans and research.

Keywords— Reinforcement learning, Machine design, PPO, pipeline design, artificial intelligence

I. INTRODUCTION

The scope for applying AI optimisation methods is very broad. Applications can be found ranging from robotics to intelligent control to the design of load-bearing structures. The algorithm must be directly adapted and its parameters tuned for any given application. Sometimes it is difficult to establish a fixed boundary between the different types of applications due to their large modularity. A common feature is their iterative flow, where a gradual evolution takes place across iterations. There are similarities to evolutionary development where there is a gradual adaptation to the environment in nature. The main difference from classical optimisation methods is the possibility of accepting even a worse solution to the next iteration. This feature may seem inefficient at first sight, but the opposite is true. The big advantage is a certain robustness and a significantly lower tendency to get stuck in a local extreme. [1, 2, 3]

Neural Networks (NNs) are one of the most well-known areas of machine learning today. NNs are structures that enable distributed parallel processing of data. The model for their creation was inspired by the behaviour of neurons in the brain. A lot of terminology is taken from this field although the actual principle of operation varies. They consist of models of neurons and the connections between them.

Individual neurons are arranged in layers. The graph showing the interconnections of the whole system is called the network topology. [4] The main parts and principles are discussed in the next few paragraphs.

A. Neuron

A neuron is a basic unit of NN. In general, it accepts several inputs, which are always assigned a certain weight that basically determines their importance. The neuron calculates a weighted sum of the inputs with added biases. The result is fed into an activation function to determine the output. There is a variety of artificial neuron models.

The basic McCulloch and Pitts model of the activation function f :

$$y = f\left(\sum_{i=1}^N (w_i x_i) - b\right)$$

Where x_i are the neuron inputs, w_i are the weights of each synapse, b is the bias, y is the output of the neuron [3, 4].

B. Activation function

The activation function allows complex relations to be modelled. It is a mathematical function that determines the output of the perceptron based on the inputs. Different activation functions are used for different purposes, and in general it is not possible to say which is better or worse for a particular application. ReLU and Softplus are the most commonly used functions.

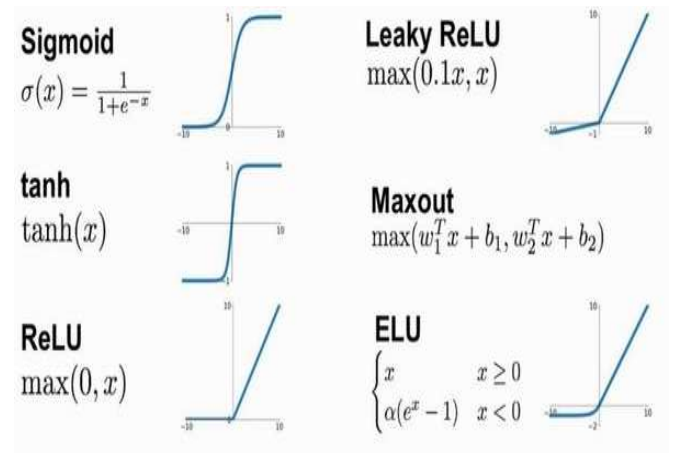


Fig. 1: Different activation functions [4]

C. Layer

Neurons are organised into several layers. The first is the input layer, which contains neurons that receive input data. These can be different types of data: numbers, images, text, etc. They are also neurons, so their weight, and with it their "importance", changes during the training process. Another term is the output layer that produces the final outputs of the network; again it is a neuron that collects data from the previous layer and produces output data based on the activation function. The layers in between are called the hidden layers. They transform input data to an output. The neurons are interconnected in complex ways, which allows calculations essential in the training. They enable the model to learn and adapt to various datasets. Without these layers, the neural network would not be able to accurately analyze and interpret information. A schematic layout of a model neural network can be seen in Figure 2.

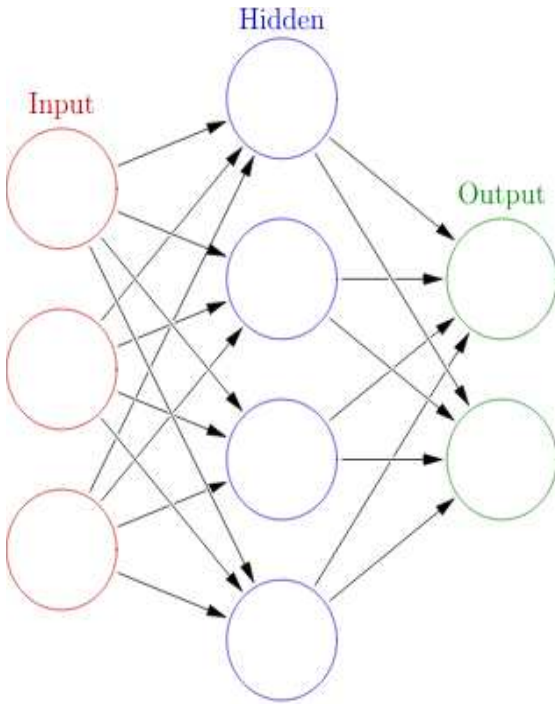


Fig. 2: Example of a neural network [6]

D. Forward propagation

As is already clear from the figure, information gradually propagates through the network from the inputs to the outputs. In each connection and neuron, the value of the information is recalculated depending on the values of the activation function and the instantaneous sum of the values for that part of the network. This process gradually results in the final output of the network. The final output of the network is the result of combining and processing the information from all parts of the network. The process of information propagation and calculation in a neural network is what allows it to learn from the data and make decisions without being explicitly programmed to do so.

E. Backpropagation

Backpropagation is an algorithm that is used to adjust the values of weights and biases to improve the behaviour of the network with respect to known data. This process is called network training. During backpropagation, the output is compared to the actual values using a so-called loss function. The resulting force is then projected (propagated) through the

network and the individual weights and biases for all neurons and connections are adjusted accordingly. By iteratively adjusting the weights and biases in this manner, the neural network can learn to better approximate the desired output for a given input. Through this process of training, the network improves its ability to make accurate predictions and classifications based on the data provided. This iterative process continues until the neural network reaches a desirable level of accuracy and performance. The success of the network training heavily relies on the choice of the loss function and the optimisation algorithm used to adjust the weights and biases.

F. Loss function

The Loss Function compares the results from the neural network with the actual data. The main objective is then to minimise this function and thus increase the accuracy of the model. This minimisation of the difference between the computation and the data works generally across tasks. Overall, the methods for computing the loss function can be divided into those for regression tasks and those for classification tasks.

The MSE (Mean Squared Error) loss function is the sum of the squared differences between the actual values and the predicted values divided by the number of values. It can be written mathematically as:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - y_{pi})^2$$

Where y_i is the actual value and y_{pi} is the predicted value, N is the number of elements. MSE penalises large errors quadratically, which means that larger errors have a larger impact on the total loss.

Another similarly important type of loss function is MAE (Mean Absolute Error):

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - y_{pi}|$$

MAE measures the average of absolute errors and is less sensitive to outliers than MSE. [7]

G. Optimisation algorithm

After computing the loss function across the neural network, we proceed to the next step, which is its actual minimisation by changing the weight and shift parameters. For this purpose, so-called optimisation functions or optimisers are deployed. Their task is to quickly find a suitable combination of parameters. The process of finding the ideal network parameters is called learning. Neural network learning can be divided into supervised learning, unsupervised learning and reinforcement learning.

H. Reinforcement learning

Reinforcement Learning (RL) uses the response of an individual (an 'agent' - a representation of the current neural network settings) to the dynamic environment to change the network parameters. Reinforcement Learning differs from supervised and unsupervised learning because it does not need labelled data or explicitly look for relationships between inputs and outputs. Instead, it focuses on finding a balance between exploration (of unknown territory) and exploitation (of current knowledge) to maximise long-term reward. In [8], the Markov Decision Process (MDP), a common dynamic

programming technique, is used to describe the environment. [9] The principle of Reinforcement Learning is easy to understand using the example in Figure 3. The mouse, which represents an agent, is trying to survive in a maze. Its task is to progressively search and explore the environment and maximise the amount of reward, in this case a piece of food. It gradually acquires some awareness of the way it moves in the maze and thus adjusts the parameters of its movement, which we can think of as the parameters of a neural network.

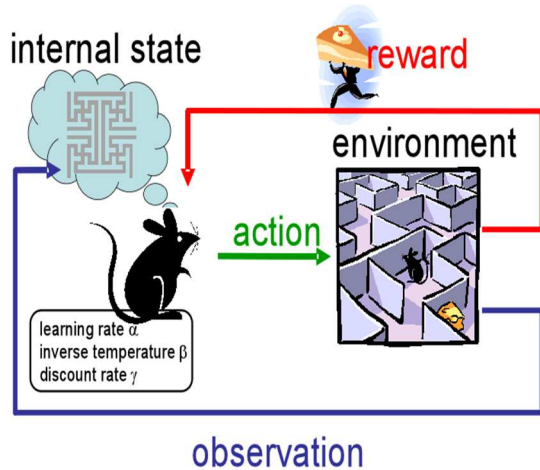


Fig. 3: Reinforcement learning – illustration [10]

The goal of an agent is to maximise the cumulative reward from a dynamic environment. Therefore, RL is often used for optimisation problems where there is a dilemma between long-term and short-term gains.

A technical system, just like a mouse in a maze, must interact with its environment, so the designer's task is to set its parameters so it fulfils its function as well as possible.

II. LITERATURE REVIEW

The traditional pathfinding methods (Dijkstra's algorithm and A* search) have been widely used in various applications. [11, 12] However, they often struggle with complex dynamic environments where there are a lot of constraints. RL handles such environments much better and can adapt to changing conditions in real-time. By using reinforcement learning, agents can make decisions based on their own experiences and interactions with the environment, rather than relying solely on predefined rules or heuristics. However, on the other hand, it comes with a price in the form of high computational cost, problematic finetuning and sometimes complicated definition of the reward functions.

RL is a relatively new method for solving such tasks, however, it has already been successfully used in several applications dedicated to path finding and pipe design. The ideal layout of pipelines is a problem in a variety of industrial fields. The efficiency and effectiveness of pipelines depend on various factors such as the terrain, the type of material being transported and the required flow rate. Engineers must carefully consider these factors when designing the layout to ensure optimum performance and minimum maintenance. Their design is often tedious and error-prone, and this is one reason why the automation of the process is required in many industrial fields. Yang et al. (2023) explored in their paper the applicability of deep reinforcement learning on the automation of building piping layout. Their research found that the use of deep reinforcement learning significantly

improved the efficiency and accuracy of pipeline layout design. Their RL algorithm can produce a reliable pipeline layout in a much shorter time than traditional algorithms [13].

The authors of another paper [14] tried to solve basically the same problem. Their application designs pipelines in plantrooms. Their algorithm which can design multiple pipelines without clashes was compared to algorithms based on more conventional approaches such as Dijkstra, A* and fruit fly.

Another example is dedicated to pipe routing in ships. The study aimed to minimise the length of the pipes and the number of bends as an objective function. Finally, the proposed methodology was verified by comparing it with existing studies that used the A*, jump point search (JPS) and other reinforcement-learning algorithms to determine the search speed, number of bends, and length of the path. The results showed that the proposed methodology outperformed the existing studies in terms of search speed, with a significant reduction of the number of bends required to reach the destination. Additionally, the path generated by the new algorithm was found to be shorter than those produced by the A*, JPS, and reinforcement-learning algorithms. These findings demonstrate the effectiveness and efficiency of the proposed methodology for pathfinding in various scenarios. [15]

III. METHODOLOGY

The aim of this paper is to apply reinforcement learning on pathfinding for general piping in machines. The algorithm should minimise the length of a given path, the number of elbows (bends) and also avoid any forbidden zones presented by the parts of the machine. It should combine the processes of machine design and optimisation.

A. Machine design

In engineering practice, optimisations often lead to problems with several conflicting requirements. A typical example is weight reduction while keeping stiffness within certain limits. In cases where the different objective functions are contradictory, the Pareto frontier is plotted. This is a set of points that represent the best combinations of solutions that can be obtained when the solution parameters are combined. Thus, it is not possible to decrease the value of one objective function without simultaneously increasing the value of another. This means that solutions on the Pareto frontier represent qualitatively comparable results.

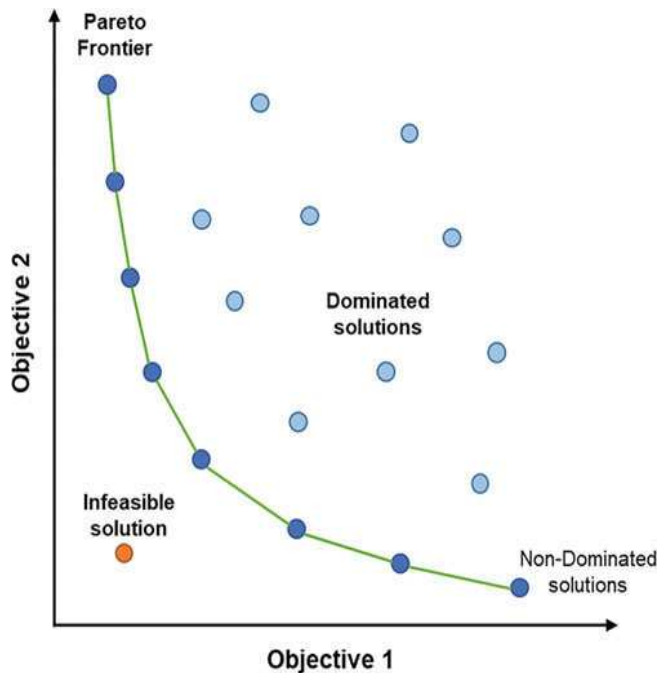


Fig. 4: Pareto frontier [16]

One of the biggest challenges in implementing evolutionary algorithms in practice is to correctly describe the function and components of individual machines. The description and analysis of engineering system requirements, or the knowledge support for the design of engineering systems, is a discipline for which the name Engineering Design Science and Methodology (EDSM) has been adopted. Technical systems (TS) do not operate in isolation but are located in a particular environment that acts on them actively or reactively. Thus, the requirements for a given TS must also be specified with respect to potential secondary inputs or outputs.

B. Application of RL

The reward function of the algorithm represents the quality of the TS, in our case the pipeline route. The program was written in Python with the help of stablebaseline3 library, which is a set of implementations of RL algorithms in PyTorch on the top of the Gym environment [17, 18]. It supports both discrete and continuous action spaces. The authors focused on simplicity and clean code, which made this library a perfect tool for common RL optimisation tasks.

The role of the optimisation algorithm was conducted by the Proximal Policy Optimisation (PPO) algorithm. It is a state-of-the-art RL algorithm, and is currently one of the most commonly used. The core idea behind PPO is to carefully update the policy to avoid any drastic changes that might lead to instability. This is achieved by using a clipped surrogate objective function. [19]

1) The environment

The environment is a description of the agent's external environment. The state is a representation of the current state of the environment as perceived by the agent. It is the basic information for its decision making. It is usually modelled as Markovian. A state is Markovian if the probability of the next state based only on the current state is the same as the probability of the next state based on the entire history.

$$P[S_{t+1}|S_t] = P[S_{t+1}|S_1, \dots, S_t]$$

P – probability, S_t – current state, S_{t+1} – future state

The current state, to put it another way, can represent the whole history. For the RL problem the state of the environment is Markovian and the history is also Markovian.

The environment of this task is represented by a grid which divides the suggested space into points. The algorithm must navigate through the grid using a series of predetermined movements from the defined start point to the end point and avoid all of the forbidden points.

2) The agent

The RL agent represents the current solution and expresses a set of algorithms responding to the environment. It contains three basic parts: policy, value function and model. The policy specifies the way the agent will behave in the environment. It can be divided into deterministic and stochastic based on the way decisions are made. The role of the value function is to predict future reward, environmental response, and future state. The model is a representation of the environment seen from the agent's perspective. However, there are also agents that do not build a model at all and operate entirely on the basis of a policy or value function.

3) The movements

There are six possible movements in the grid world: up, down, left, right, forwards and backwards. These movements represent a certain section of the pipeline. If the two movements in the path are the same, they represent a straight pipe. Otherwise, there must be a bend.

4) The reward function

RL works on the principle of trial and error, by which it gradually searches the environment. In this way, the agent gradually adjusts its parameters. For each task, there is an important decision between exploration and exploitation. The goal of exploration is to find out more about the environment and potentially find better actions that can yield higher rewards in the future. Exploitation, on the other hand, uses the information already obtained about the environment to obtain the highest possible reward. Successfully balancing exploration and exploitation is the key to achieving effective learning. The reward is calculated after every move for the current state of the agent. It depends on the distance to the final point, forbidden zones, sequence of movements, and its length. Ultimately, a well-balanced approach to exploration and exploitation will lead to more efficient learning and improved decision-making in navigating the environment.

The definition of the reward function could be very tricky and easily lead to incorrect results. The obvious reward is for reaching the goal position. This is 100 in our case. However, setting just this reward may not be enough, because the agent has to go unguided through many steps to get it. That is why more immediate functions must be introduced. The agent receives the positive reward +1 if it gets closer to the final position and the negative reward -1 if it gets further away. In addition, a basic punishment for each step is also implemented. This is -0.5 in our case. The last is the implementation of a negative reward for a collision with an obstacle of -20. It was also necessary to introduce a function that limits the number of possible steps, because without it the algorithm could search the solution space for a very long time in every iteration. Another important reward was

implemented to minimise the number of pipe elbows. The agent receives a penalty of -10 if it makes an elbow and a reward of +5 if it holds its direction.

IV. RESULTS

The RL algorithm proved to be an efficient solution for designing pipelines. It could automate the tedious task of path planning in most machines. However, there are still disadvantages. The learning phase can be quite long, especially for bigger grids. This limits the capabilities of the algorithm and its functionality for application to bigger machines. The main problem was the combination of the successful goal point search and minimisation of the number of bends. The algorithm was tested for three different sets, represented by the cubes of points. The first point was always set to $[0,0,0]$ and the goal point was in the point across the solution space cube. The testing of the forbidden zones was in the shape of a cross with two rectangular holes in it. A crash into them meant the end of the agent pathfinding

The representation of the path from the start point $[0,0,0]$ to $[9,9,9]$ is shown in Figure 5.

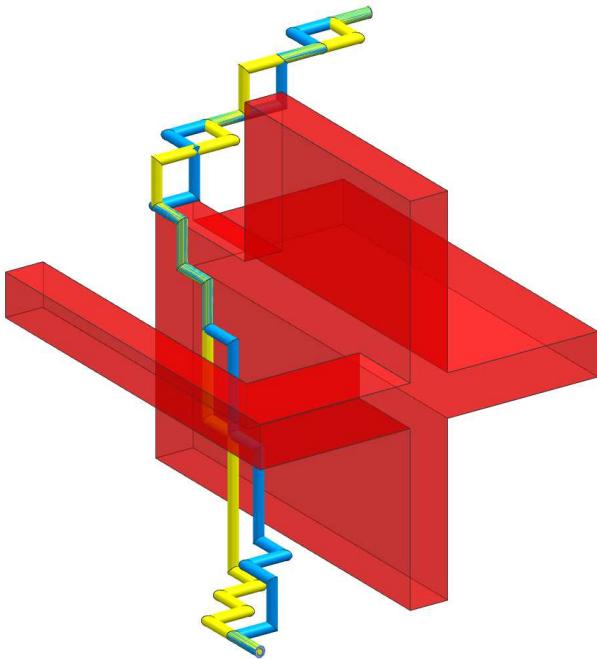


Fig. 5: Paths in a 10 x 10 x 10 cube

The blue line represents the solution after 300,000 iterations. It has 21 elbows. The net training took 21 minutes. The yellow line is the representation of the solution with 500,000 iterations. It has 18 elbows and took approximately 27 minutes.

The representation of the path from start point $[0,0,0]$ to $[19,19,19]$ is shown in the next figure (Fig.6).

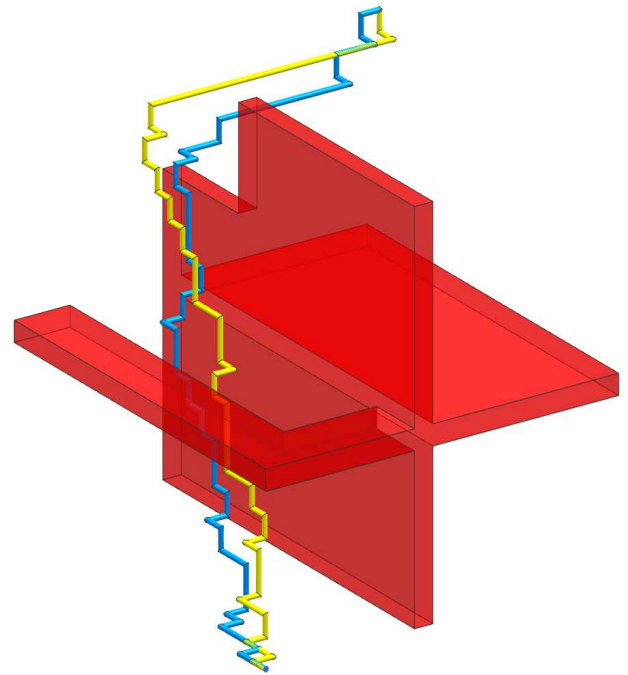


Fig. 6: Paths in a 20 x 20 x 20 cube

The calculation of the blue solution with 300,000 iterations took 17 minutes. It has 38 elbows. The next solution with 500,000 iterations took 31 minutes and has 35 elbows.

The representation of the path from start point $[0,0,0]$ to $[49,49,49]$ is shown in the next figure (Fig.7).

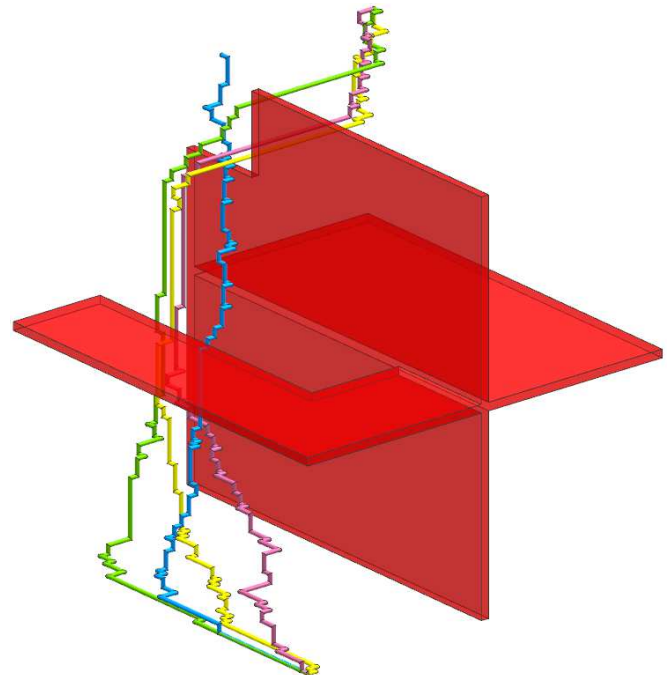


Fig. 7: Paths in a 50 x 50 x 50 cube

The first calculation (the blue line) was set to 300,000 iterations. However, this was not sufficient to reach the goal point. The calculation took approximately 18 minutes. The next solution (the yellow line) took about 40 minutes and consisted of 700,000 iterations. The algorithm was able to reach the goal point with 60 elbows. The number of iterations of the next solution (green line?) was set to 1,100,000. It took about 55 minutes to calculate and was able to reach the goal.

It consists of 44 elbows. The last solution (pink) tested the algorithm capabilities with a high number of iterations. This number was set to 15,000,000. The calculation took 15 hours and led to a solution with 77 elbows, which is worse than previous calculation. The model was overfitted, capturing noise and random fluctuations in the training data. Overfitting typically occurs when the model is overly complex, with too many parameters relative to the training data, or when training continues without proper regularization.

V. CONCLUSION

The algorithm proved to be efficient in the limited, but sufficient, range of tasks. Therefore, it is possible to use it for the design of certain machines, however, its designs are not as good as those created by human designers. The authors would like to continue the work and make the algorithm more robust and capable of achieving better results. Another goal is to scale the possible grid to a greater extent without a significant increase in the required computational power. The most important task will be the improvement of the agent's perception of its surroundings. It currently works just with the distance to the goal point, a reward for its position and the current vector of the pipeline. Understanding the distance to the closest obstacle/grid border could significantly help the agent to make a decision and thus speed up the whole process and acquire solutions with fewer elbows.

Another important step is the development of the automatic connection with a CAD program and the automatic creation of pipe parts and an assembly. The final goal is to wrap it up under one button and include it alongside common user functions in CAD.

VI. FUNDING

This research was funded by University of West Bohemia, grant number SGS-2023-025.

VII. DECLARATIONS

A. Conflict of interest

The authors have no conflicts of interest to declare that are relevant to the content of this article.

VIII. REFERENCES

- [1] I. Zelinka, *Evoluční výpočetní techniky: principy a aplikace*. 2009. ISBN 978-80-7300-218-3
- [2] I. Zelinka, *Umělá inteligence v problémech globální optimalizace*. 2002. ISBN 80-730-0069-5
- [3] V. Mařík, O. Štěpánková, and J. Lažanský, *Umělá inteligence – 3. díl*. 2001. ISBN 80-200-0472-6
- [4] V. Mařík, O. Štěpánková, and J. Lažanský, *Umělá inteligence*. 1993. ISBN 80-200-0496-3
- [5] Rahul Jayawardana and Sameera Bandaranayake, "ANALYSIS OF OPTIMIZING NEURAL NETWORKS AND ARTIFICIAL INTELLIGENT MODELS FOR GUIDANCE, CONTROL, AND NAVIGATION SYSTEMS," *International Research Journal of Modernization in Engineering, Technology and Science*, vol. 3, no. 3, Art. no. e-ISSN: 2582-5208, Mar. 2021.
- [6] "Neural network (machine learning)." Aug. 23, 2024. Accessed: Sep. 03, 2024. [Online]. Available: [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning))
- [7] A. Oppermann, "How Loss Functions Work in Neural Networks and Deep Learning," <https://builtin.com/>, Dec. 14, 2022. <https://builtin.com/machine-learning/loss-functions> (accessed Sep. 03, 2024).
- [8] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement Learning: a survey," *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, May 1996, doi: 10.1613/jair.301.
- [9] M. Van Otterlo and M. Wiering, "Reinforcement learning and Markov decision processes," in *Adaptation, learning, and optimization*, 2012, pp. 3–42. doi: 10.1007/978-3-642-27645-3_1.
- [10] Aneek Das, "The very basics of Reinforcement Learning," *Becoming Human: Artificial Intelligence Magazine*, <https://medium.com/>, Mar. 26, 2017. [Online]. Available: <https://becominghuman.ai/the-very-basics-of-reinforcement-learning-154f28a79071>
- [11] H. M. Dbouk, H. Hayek, and K. Ghorayeb, "Modular approach for optimal pipeline layout," *Journal of Petroleum Science and Engineering*, vol. 197, p. 107934, Feb. 2021, doi: 10.1016/j.petrol.2020.107934.
- [12] F.-X. Chu and S.-Y. Chen, "Optimal design of pipeline based on the shortest path," *Physics Procedia*, vol. 33, pp. 216–220, Jan. 2012, doi: 10.1016/j.phpro.2012.05.054.
- [13] C. Yang, Z. Zheng, and J.-R. Lin, "Automatic design method of building pipeline layout based on deep reinforcement learning," *arXiv (Cornell University)*, Jan. 2023, doi: 10.48550/arxiv.2305.10760.
- [14] J. Singh and J. C. P. Cheng, "Automating the generation of 3D multiple pipe layout design using BIM and heuristic search methods," in *Lecture notes in civil engineering*, 2020, pp. 54–72. doi: 10.1007/978-3-030-51295-8_6.
- [15] Y. Kim, K. Lee, B. Nam, and Y. Han, "Application of reinforcement learning based on curriculum learning for the pipe auto-routing of ships," *Journal of Computational Design and Engineering*, vol. 10, no. 1, pp. 318–328, Jan. 2023, doi: 10.1093/jcde/qwad001.
- [16] X. Liu, A. P. IJzerman, and G. J. P. Van Westen, "Computational Approaches for de novo drug Design: past, present, and future," *Methods in Molecular Biology*, pp. 139–165, Aug. 2020, doi: 10.1007/978-1-0716-0826-5_6.
- [17] Raffin, Hill, Gleave, Kanervisto, Ernestus, and Dormann, "Stable-Baselines3 Docs - Reliable Reinforcement Learning Implementations," stable-baselines3.readthedocs.io, Nov. 21AD. <https://stable-baselines3.readthedocs.io/en/master/#stable-baselines3-docs-reliable-reinforcement-learning-implementations> (accessed Aug. 20, 2024).
- [18] "An API standard for reinforcement learning with a diverse collection of reference environments," gymnasium.farama.org. <https://gymnasium.farama.org/> (accessed Aug. 20, 2024).
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy optimization Algorithms," *arXiv (Cornell University)*, Jan. 2017, doi: 10.48550/arxiv.1707.06347.